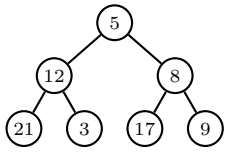
Assignment 2

MyungJin Suh EECE 7205, Fall 2026



arr = {5, 12, 8, 21, 3, 17, 9}, $n = 7$, $h = 2$

Counting conventions. Every numbered line costs 1 and is counted each time it runs (a test each time it is evaluated). The **procedure** header, **else** and lines that only end a block are not counted, nor is the initial call (as in the Merge/Quick Sort handouts); a recursive call is counted once on the line that makes it. A **for** loop gives three rows: initialization, test, increment (as in the Selection Sort handout). A tree with n nodes has $2n$ child pointers, $n-1$ of them edges, so $n+1$ NULL pointers; a recursive procedure called on every node and every NULL child is therefore called $1 + 2n = 2n + 1$ times (once initially, twice by each real node).

Question 1 (5 points) Maximum depth of a binary tree

Idea. Depth below a node = 1 + the depth of its deeper subtree; an empty subtree has depth -1, so a leaf gets $1 + \max(-1, -1) = 0$. Both subtrees answer first (postorder).

(a) Pseudocode

Algorithm 1 Maximum Depth

Input: pointer node to the root of a binary tree
Output: edges on the longest root-to-leaf path (-1 if empty)
Initial call: MaxDepth(root)

```
1: procedure MaxDepth(node)
2:     if node = NULL then
3:         return −1
4:     end if
5:     leftDepth ← MaxDepth(node.left)
6:     rightDepth ← MaxDepth(node.right)
7:     return 1 + max(leftDepth, rightDepth)
8: end procedure
```

(b) Worked example

Real node: lines 2, 5–7; NULL: lines 2, 3.

#	Call	Lines	Returns
1	5	2,5–7	$1 + \max(1, 1) = 2$
2	12	2,5–7	$1 + \max(0, 0) = 1$
3	21	2,5–7	$1 + \max(-1, -1) = 0$
4	NULL (21.L)	2,3	-1
5	NULL (21.R)	2,3	-1
6	3	2,5–7	$1 + \max(-1, -1) = 0$
7	NULL (3.L)	2,3	-1
8	NULL (3.R)	2,3	-1
9	8	2,5–7	$1 + \max(0, 0) = 1$
10	17	2,5–7	$1 + \max(-1, -1) = 0$
11	NULL (17.L)	2,3	-1
12	NULL (17.R)	2,3	-1
13	9	2,5–7	$1 + \max(-1, -1) = 0$
14	NULL (9.L)	2,3	-1
15	NULL (9.R)	2,3	-1

Output **2** = expected. 15 calls = $2n + 1$ (7 real, 8 NULL).

(c) Frequency count

Line	Operation	Cost	Freq.	Count
2	test node = NULL	1	$2n + 1$	15
3	return -1	1	$n + 1$	8
5	left recursive call	1	n	7
6	right recursive call	1	n	7
7	return 1 + max	1	n	7
	Total $T(n)$		$\mathbf{6n + 2}$	**44**

Check: $6(7) + 2 = 44$; trace: $7 \times 4 + 8 \times 2 = 44$. ✓

(d) Final complexity.

$T(n) = 6n + 2 = \Theta(n)$; **best** = **worst** = $\Theta(n)$, since every node and all $n+1$ NULL pointers are visited whatever the shape. **Extra space** $\Theta(h)$ (at most $h + 2$ frames): $\Theta(\log n)$ for a complete tree (4 frames here), $\Theta(n)$ worst. No input is slower in time; for space the worst input is a chain ($h = n - 1$).

Twist (chain of 7). The count does not change: a chain still has 7 real nodes (15 calls) and $n + 1 = 8$ NULL pointers, so $T(7) = 44$. Only the answer (6) and the stack depth (8 frames, $\Theta(n)$ space) change.

Question 2 (5 points) Sum of all nodes

Idea. sum = root value + the sums of both subtrees; an empty tree sums to 0. The queue version is level order (rule T5) with a running total.

(a) Pseudocode

Algorithm 2a Sum (recursive)

Input: pointer node to the root
Output: sum of all values (0 if empty)
Initial call: Sum(root)

```
1: procedure Sum(node)
2:     if node = NULL then
3:         return 0
4:     end if
5:     leftSum ← Sum(node.left)
6:     rightSum ← Sum(node.right)
7:     return node.data + leftSum + rightSum
8: end procedure
```

(b) Worked example

2a: real node lines 2, 5–7; NULL lines 2, 3.

#	Call	Lines	Returns
1	5	2,5–7	$5 + 36 + 34 = 75$
2	12	2,5–7	$12 + 21 + 3 = 36$
3	21	2,5–7	$21 + 0 + 0 = 21$
4	NULL (21.L)	2,3	0
5	NULL (21.R)	2,3	0
6	3	2,5–7	$3 + 0 + 0 = 3$
7	NULL (3.L)	2,3	0
8	NULL (3.R)	2,3	0
9	8	2,5–7	$8 + 17 + 9 = 34$
10	17	2,5–7	$17 + 0 + 0 = 17$
11	NULL (17.L)	2,3	0
12	NULL (17.R)	2,3	0
13	9	2,5–7	$9 + 0 + 0 = 9$
14	NULL (9.L)	2,3	0
15	NULL (9.R)	2,3	0

Algorithm 2b Sum (queue, level order)

Input: pointer root to the root
Output: sum of all values (0 if empty)
Initial call: SumQueue(root)

```
 1: procedure SumQueue(root)
 2:     if root = NULL then
 3:         return 0
 4:     end if
 5:     total ← 0
 6:     Q ← empty queue
 7:     Enqueue(Q, root)
 8:     while Q is not empty do
 9:         node ← Dequeue(Q)
10:         total ← total + node.data
11:         if node.left ≠ NULL then
12:             Enqueue(Q, node.left)
13:         end if
14:         if node.right ≠ NULL then
15:             Enqueue(Q, node.right)
16:         end if
17:     end while
18:     return total
19: end procedure
```

2b, one row per pass (queue front first):

Pass	Queue before	Deq.	total	Enqueue	Queue after
1	5	5	0+5=5	12, 8	12, 8
2	12, 8	12	5+12=17	21, 3	8, 21, 3
3	8, 21, 3	8	17+8=25	17, 9	21, 3, 17, 9
4	21, 3, 17, 9	21	25+21=46	none	3, 17, 9
5	3, 17, 9	3	46+3=49	none	17, 9
6	17, 9	17	49+17=66	none	9
7	9	9	66+9=75	none	empty

Both return **75** = expected; line 8 is tested an 8th time and stops.

(c) Frequency count

2a (same rows as Algorithm 1):

Line	Operation	Cost	Freq.	Count
2	test node = NULL	1	$2n+1$	15
3	return 0	1	$n+1$	8
5	left recursive call	1	n	7
6	right recursive call	1	n	7
7	return the sum	1	n	7
	Total		**$6n+2$**	**44**

2b: $n_L + n_R = n - 1$ (every node but the root is enqueued once).

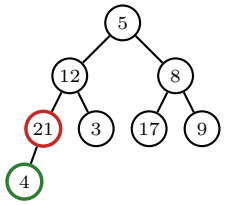
Line	Operation	Cost	Freq.	Count
2	test root = NULL	1	1	1
5	total ← 0	1	1	1
6	Q ← empty queue	1	1	1
7	Enqueue(Q, root)	1	1	1
8	test Q not empty	1	$n+1$	8
9	node ← Dequeue(Q)	1	n	7
10	total ← total + data	1	n	7
11	test left ≠ NULL	1	n	7
12	Enqueue left	1	n_L	3
14	test right ≠ NULL	1	n	7
15	Enqueue right	1	n_R	3
18	return total	1	1	1
	Total		**$6n+5$**	**47**

(d) Final complexity. Both $\Theta(n)$ in the **best** and **worst** case: each node is processed once whatever the shape. **Extra space:** 2a $\Theta(h)$ (stack), 2b $\Theta(w)$ (queue, w = max width); both peak at 4 here. Worst input: a chain for 2a, a complete tree (last level $\approx n/2$) for 2b.

Twist. Time is almost the same ($6n+2$ vs. $6n+5$: three set-up lines). Space goes opposite ways: a complete tree has $h = \lfloor \log_2 n \rfloor$ but $w \approx (n+1)/2$, so recursion $\Theta(\log n)$ vs. queue $\Theta(n)$; a chain has $h = n-1$, $w = 1$, so recursion $\Theta(n)$ vs. queue $\Theta(1)$.

Question 3 (5 points) Count the nodes that have exactly one child

Input. INSERT(root, 4): with $n = 7$, rule I6 puts 4 at index 7, parent $(7-1)/2 = 3$ (node 21); 7 is odd, so 4 is the *left* child (I3). Now $n = 8$, level order 5, 12, 8, 21, 3, 17, 9, 4.
Idea. Count one-child nodes in both subtrees, then add 1 if exactly one of the node's two pointers is NULL (postorder).



(a) Pseudocode

Algorithm 3 Nodes with exactly one child

Input: pointer node to the root
Output: number of nodes with exactly one child
Initial call: OneChild(root)

```
 1: procedure OneChild(node)
 2:     if node = NULL then
 3:         return 0
 4:     end if
 5:     leftCount ← OneChild(node.left)
 6:     rightCount ← OneChild(node.right)
 7:     if (node.left = NULL) xor (node.right = NULL) then
 8:         return 1+ leftCount + rightCount
 9:     end if
10:     return leftCount + rightCount
11: end procedure
```

(b) Worked example

On the 8-node tree.

#	Call	Lines	Returns
1	5	2,5–7,10	1 + 0 = 1
2	12	2,5–7,10	1 + 0 = 1
3	21	2,5–8	1 + 0 + 0 = 1
4	4	2,5–7,10	0 + 0 = 0
5	NULL (4.L)	2,3	0
6	NULL (4.R)	2,3	0
7	NULL (21.R)	2,3	0
8	3	2,5–7,10	0 + 0 = 0
9	NULL (3.L)	2,3	0
10	NULL (3.R)	2,3	0
11	8	2,5–7,10	0 + 0 = 0
12	17	2,5–7,10	0 + 0 = 0
13	NULL (17.L)	2,3	0
14	NULL (17.R)	2,3	0
15	9	2,5–7,10	0 + 0 = 0
16	NULL (9.L)	2,3	0
17	NULL (9.R)	2,3	0

Output **1** (node 21) = expected; 17 calls = $2n+1$.

(c) Frequency count k = one-child nodes ($k = 1$); each real node returns via line 8 or 10, so k cancels.

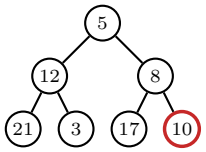
Line	Operation	Cost	Freq.	Count (n=8)
2	test node = NULL	1	$2n+1$	17
3	return 0	1	$n+1$	9
5	left recursive call	1	n	8
6	right recursive call	1	n	8
7	test exactly one child	1	n	8
8	return $1+\ldots$	1	k	1
10	return $\ldots$	1	$n-k$	7
	Total		$\mathbf{7n+2}$	**58**

(d) Final complexity. $T(n) = 7n + 2 = \Theta(n)$, **best** = **worst** = $\Theta(n)$: every node must be inspected. **Extra space** $\Theta(h)$; a chain is the worst input for space.

Twist. In an array-built tree node i has a left child iff $2i+1 \le n-1$ and a right child iff $2i+2 \le n-1$. Exactly one child iff $2i+1 \le n-1 < 2i+2$, i.e. $n = 2i+2$. For even $n \ge 2$ the only solution is $i = n/2 - 1$ (one node); for odd n there is none because $2i+2$ is even. Here $n = 8$ gives $i = 3$, node 21; before the insertion $n = 7$ and the count was 0.

Question 4 (5 points) Are two trees identical?

Idea. Walk both trees in preorder in lockstep: two NULLs match; one NULL means the shapes differ; different values mean the trees differ; otherwise compare the left pair, and the right pair only if the left matched. The first difference ends the walk.



tree C

(a) Pseudocode

Algorithm 4 Identical trees

Input: roots a, b of two binary trees
Output: true iff same shape and same values
Initial call: IDENTICAL(rootA, rootB)

```
 1: procedure IDENTICAL(a, b)
 2:     if a = NULL and b = NULL then
 3:         return true
 4:     end if
 5:     if a = NULL or b = NULL then
 6:         return false
 7:     end if
 8:     if a.data ≠ b.data then
 9:         return false
10:     end if
11:     if IDENTICAL(a.left, b.left) = false then
12:         return false
13:     end if
14:     return IDENTICAL(a.right, b.right)
15: end procedure
```

(b) Worked example

IDENTICAL(A, B) = **true**:

#	Pair	Lines	Returns
1	(5, 5)	2,5,8,11,14	true
2	(12, 12)	2,5,8,11,14	true
3	(21, 21)	2,5,8,11,14	true
4	(N, N) (21.L)	2,3	true
5	(N, N) (21.R)	2,3	true
6	(3, 3)	2,5,8,11,14	true
7	(N, N) (3.L)	2,3	true
8	(N, N) (3.R)	2,3	true
9	(8, 8)	2,5,8,11,14	true
10	(17, 17)	2,5,8,11,14	true
11	(N, N) (17.L)	2,3	true
12	(N, N) (17.R)	2,3	true
13	(9, 9)	2,5,8,11,14	true
14	(N, N) (9.L)	2,3	true
15	(N, N) (9.R)	2,3	true

IDENTICAL(A, C) = **false** (same first 12 calls; call 13 finds $9 \neq 10$ and the false is passed up through line 14):

#	Pair	Lines	Returns
1	(5, 5)	2,5,8,11,14	false
2	(12, 12)	2,5,8,11,14	true
3	(21, 21)	2,5,8,11,14	true
4	(N, N) (21.L)	2,3	true
5	(N, N) (21.R)	2,3	true
6	(3, 3)	2,5,8,11,14	true
7	(N, N) (3.L)	2,3	true
8	(N, N) (3.R)	2,3	true
9	(8, 8)	2,5,8,11,14	false
10	(17, 17)	2,5,8,11,14	true
11	(N, N) (17.L)	2,3	true
12	(N, N) (17.R)	2,3	true
13	(9, 10)	2,5,8,9	false ($9 \neq 10$)

Both results match the expected output.

(c) Frequency count

worst case = identical trees.

Line	Operation	Cost	Freq.	(A,B)	(A,C)
2	test both NULL	1	$2n+1$	15	13
3	return true	1	$n+1$	8	6
5	test one NULL	1	n	7	7
6	return false	1	0	0	0
8	test data differ	1	n	7	7
9	return false	1	0	0	1
11	test left pair	1	n	7	6
12	return false	1	0	0	0
14	return right pair	1	n	7	6
	Total		$\mathbf{7n+2}$	**51**	**46**

(d) Final complexity. **Worst** $\Theta(n)$ $(7n+2)$: identical trees, or trees that differ only at the last pair in preorder, since every pair must be compared. **Best** $\Theta(1)$: the roots differ (lines 2, 5, 8, 9). **Extra space** $\Theta(h)$. **Twist.** IDENTICAL(A, C) first finds a difference at call 13, the pair (9, 10) under 8: the *last* node in preorder (5, 12, 21, 3, 8, 17, 9), so it costs 46 of the 51 lines of a full match (it skips the two NULL pairs under 9 and lines 11, 14 there, but runs line 9). For two trees of n nodes: best $\Theta(1)$ (roots differ), worst $\Theta(n)$ (identical).

Question 5 (5 points) Minimum value in a binary tree

Idea. The tree is unordered, so take the min of the node's value and both subtree minima; NULL returns $+\infty$, which never wins.

(a) Pseudocode

Algorithm 5 Minimum value

Input: pointer node to the root
Output: smallest value ($+\infty$ if empty)
Initial call: MinValue(root)

```
1: procedure MinValue(node)
2:     if node = NULL then
3:         return +∞
4:     end if
5:     leftMin ← MinValue(node.left)
6:     rightMin ← MinValue(node.right)
7:     return min(node.data, leftMin, rightMin)
8: end procedure
```

(b) Worked example

#	Call	Lines	Returns
1	5	2,5–7	$\min(5,3,8)=3$
2	12	2,5–7	$\min(12,21,3)=3$
3	21	2,5–7	$\min(21,\infty,\infty)=21$
4	NULL (21.L)	2,3	$+\infty$
5	NULL (21.R)	2,3	$+\infty$
6	3	2,5–7	$\min(3,\infty,\infty)=3$
7	NULL (3.L)	2,3	$+\infty$
8	NULL (3.R)	2,3	$+\infty$
9	8	2,5–7	$\min(8,17,9)=8$
10	17	2,5–7	$\min(17,\infty,\infty)=17$
11	NULL (17.L)	2,3	$+\infty$
12	NULL (17.R)	2,3	$+\infty$
13	9	2,5–7	$\min(9,\infty,\infty)=9$
14	NULL (9.L)	2,3	$+\infty$
15	NULL (9.R)	2,3	$+\infty$

Output **3** = expected.

(c) Frequency count

Line	Operation	Cost	Freq.	Count
2	test node = NULL	1	$2n+1$	15
3	return $+\infty$	1	$n+1$	8
5	left recursive call	1	n	7
6	right recursive call	1	n	7
7	return the min	1	n	7
	Total		$\mathbf{6n+2}$	**44**

(d) Final complexity. $\Theta(n)$ **best** and **worst**: without order any unvisited node could be smaller. **Extra space** $\Theta(h)$, worst for a chain.

Algorithm 5b Minimum in a BST

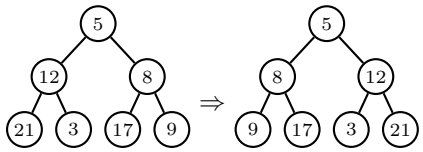
Input: root of a non-empty BST
Output: smallest value
Initial call: BSTMin(root)

```
1: procedure BSTMin(root)
2:     node ← root
3:     while node.left ≠ NULL do
4:         node ← node.left
5:     end while
6:     return node.data
7: end procedure
```

Twist (BST). In a BST the minimum is the *leftmost* node: follow left pointers, no recursion. With d the depth of the leftmost node, line 2 runs once, line 3 $d+1$ times, line 4 d times, line 6 once: $T = 2d+3$. On S $= \{9,5,17,3,8,12,21\}$ (Q14): $9 \to 5 \to 3$, $d=2$, $T=7$, returns 3. Cost becomes $\Theta(h)$: best $\Theta(1)$ (no left child at the root), $\Theta(\log n)$ balanced, worst $\Theta(n)$ (left chain); extra space $\Theta(1)$.

Question 6 (7 points) Invert (mirror) a binary tree

Idea. Invert both subtrees, then attach the inverted right subtree on the left and the inverted left subtree on the right (postorder). Each node's two pointers are swapped exactly once; NULL children stay NULL.



(a) Pseudocode

Algorithm 6 Invert a binary tree

Input: pointer node to the root
Output: root of the mirrored tree (in place)
Initial call: root ← Invert(root)

```
1: procedure Invert(node)
2:     if node = NULL then
3:         return NULL
4:     end if
5:     leftInv ← Invert(node.left)
6:     rightInv ← Invert(node.right)
7:     node.left ← rightInv
8:     node.right ← leftInv
9:     return node
10: end procedure
```

(b) Worked example

Swaps happen on the way back up.

#	Call	Lines	Lines 7–8 set	Ret.
1	5	2,5–9	L←8, R←12	5
2	12	2,5–9	L←3, R←21	12
3	21	2,5–9	L←N, R←N	21
4	NULL (21.L)	2,3	–	NULL
5	NULL (21.R)	2,3	–	NULL
6	3	2,5–9	L←N, R←N	3
7	NULL (3.L)	2,3	–	NULL
8	NULL (3.R)	2,3	–	NULL
9	8	2,5–9	L←9, R←17	8
10	17	2,5–9	L←N, R←N	17
11	NULL (17.L)	2,3	–	NULL
12	NULL (17.R)	2,3	–	NULL
13	9	2,5–9	L←N, R←N	9
14	NULL (9.L)	2,3	–	NULL
15	NULL (9.R)	2,3	–	NULL

Level order after: **5, 8, 12, 9, 17, 3, 21** = expected.

(c) Frequency count

Line	Operation	Cost	Freq.	Count
2	test node = NULL	1	$2n+1$	15
3	return NULL	1	$n+1$	8
5	left recursive call	1	n	7
6	right recursive call	1	n	7
7	node.left ← rightInv	1	n	7
8	node.right ← leftInv	1	n	7
9	return node	1	n	7
	Total		$\mathbf{8n+2}$	**58**

(d) Final complexity. $T(n) = 8n + 2 = \Theta(n)$, **best = worst**: every node must be rewired. **Extra space** $\Theta(h)$ (worst: chain); the inversion is in place.

Twist. Inorder before: 21 12 3 5 17 8 9; after: **9 8 17 5 3 12 21**, exactly the reverse. Mirroring swaps L and R at every node, so inorder (L N R) of the mirror visits the original in R N L order (reverse inorder). By induction, with L, R the subtrees of the root N: $\text{in}(\text{mir}(T)) = \text{in}(\text{mir}(R))\, N\, \text{in}(\text{mir}(L)) = \text{rev}(\text{in}(R))\, N\, \text{rev}(\text{in}(L)) = \text{rev}(\text{in}(T))$.

Question 7 (7 points) Print each level on its own line, and the right side view

Idea. Level order (T5), but each pass first reads size ← SIZE(Q): the queue then holds exactly one level, which the inner loop prints while enqueuing the next. The node with i = size is the rightmost of its level.

(a) Pseudocode

Algorithm 7 Levels and right side view

```
Input: pointer root to the root
Output: each level on its own line; returns the right side view
Initial call: view ← LevelsAndView(root)
 1: procedure LevelsAndView(root)
 2:     if root = NULL then
 3:         return empty list
 4:     end if
 5:     view ← empty list
 6:     Q ← empty queue
 7:     Enqueue(Q, root)
 8:     while Q is not empty do
 9:         size ← Size(Q)
10:         for i ← 1 to size do
11:             node ← Dequeue(Q)
12:             print node.data
13:             if node.left ≠ NULL then
14:                 Enqueue(Q, node.left)
15:             end if
16:             if node.right ≠ NULL then
17:                 Enqueue(Q, node.right)
18:             end if
19:             if i = size then
20:                 Append(view, node.data)
21:             end if
22:         end for
23:         print newline
24:     end while
25:     return view
26: end procedure
```

(b) Worked example One row per pass (level).

Level	Queue before	size	Line printed	i =size	Queue after	view
1	5	1	5	5	12, 8	5
2	12, 8	2	12 8	8	21, 3, 17, 9	5, 8
3	21, 3, 17, 9	4	21 3 17 9	9	empty	5, 8, 9

Levels **[5], [12, 8], [21, 3, 17, 9]**; view **5, 8, 9** = expected.

(c) Frequency count $L = h + 1 = 3$ levels; $n_L + n_R = n - 1$.

Line	Operation	Cost	Freq.	Count
2	test root = NULL	1	1	1
5	view ← empty	1	1	1
6	Q ← empty	1	1	1
7	Enqueue(root)	1	1	1
8	test Q not empty	1	$L+1$	4
9	size ← Size(Q)	1	L	3
10	for: init / test / incr.	1	L / n+L / n	3/10/7
11	Dequeue	1	n	7
12	print node.data	1	n	7
13	test left	1	n	7
14	Enqueue left	1	n_L	3
16	test right	1	n	7
17	Enqueue right	1	n_R	3
19	test i = size	1	n	7
20	Append to view	1	L	3
23	print newline	1	L	3
25	return view	1	1	1
	Total		**8n+6L+5**	**79**

$= 8n + 6h + 11$; check $56 + 18 + 5 = 79$. ✓

Twist (where the terms come from). $8n$ is the per-node work: lines 11, 12, 13, 16, 19, the inner test and increment, and the $n - 1$ enqueues. $6L = 6(h + 1)$ is the per-level work: outer test, size, inner-loop init, its final failed test, Append to view, newline. The constant 5 is the set-up and return. **(d) Final complexity.** With $1 \le L \le n$: $8n + 6\lfloor \log_2 n \rfloor + 11$ (complete tree) $\le T(n) \le 14n + 5$ (chain, one node per level, so per-level work is paid n times). **Best = worst** $= \Theta(n)$. **Extra space** $\Theta(w)$ queue + $\Theta(h)$ view, $\Theta(n)$ worst.

Question 8 (7 points) Zigzag (spiral) level order traversal

Idea. A stack reverses what is pushed on it. S1 holds an even level (popped left to right); its nodes push children *left then right* onto S2, so S2 pops right to left. S2's nodes push *right then left* onto S1, so the next level pops left to right again.

(a) Pseudocode

Algorithm 8a Zigzag with two stacks

Input: pointer root to the root
Output: prints levels alternately L→R and R→L
Initial call: ZigzagStacks(root)

```
 1: procedure ZigzagStacks(root)
 2:     if root = NULL then
 3:         return
 4:     end if
 5:     S1 ← empty stack                          ▷ even levels
 6:     S2 ← empty stack                          ▷ odd levels
 7:     Push(S1, root)
 8:     while S1 is not empty do
 9:         while S1 is not empty do
10:             node ← Pop(S1)
11:             print node.data
12:             if node.left ≠ NULL then
13:                 Push(S2, node.left)
14:             end if
15:             if node.right ≠ NULL then
16:                 Push(S2, node.right)
17:             end if
18:         end while
19:         while S2 is not empty do
20:             node ← Pop(S2)
21:             print node.data
22:             if node.right ≠ NULL then
23:                 Push(S1, node.right)
24:             end if
25:             if node.left ≠ NULL then
26:                 Push(S1, node.left)
27:             end if
28:         end while
29:     end while
30: end procedure
```

(b) Worked example

One row per pop; stacks bottom to top.

Pop	From	Node	Push	S1 after	S2 after	Output
1	S1	5	S2←12, 8	empty	12, 8	5
2	S2	8	S1←9, 17	9, 17	12	5 8
3	S2	12	S1←3, 21	9, 17, 3, 21	empty	5 8 12
4	S1	21	S2←none	9, 17, 3	empty	5 8 12 21
5	S1	3	S2←none	9, 17	empty	5 8 12 21 3
6	S1	17	S2←none	9	empty	5 8 12 21 3 17
7	S1	9	S2←none	empty	empty	5 8 12 21 3 17 9

Output **5, 8, 12, 21, 3, 17, 9** = expected.

(c) Frequency count

$E = \lceil L/2 \rceil$ even levels = outer passes ($E = 2$); n_e, n_o nodes on even/odd levels $(5, 2)$; the four pushes total $n - 1$.

Line	Operation	Cost	Freq.	Count
2	test root = NULL	1	1	1
5	S1 ← empty	1	1	1
6	S2 ← empty	1	1	1
7	Push(S1, root)	1	1	1
8	outer test	1	$E + 1$	3
9	S1 loop test	1	$n_e + E$	7
10	Pop(S1)	1	n_e	5
11	print	1	n_e	5
12	test left	1	n_e	5
13	Push left on S2	1	ℓ_e	1
15	test right	1	n_e	5
16	Push right on S2	1	r_e	1
19	S2 loop test	1	$n_o + E$	4
20	Pop(S2)	1	n_o	2
21	print	1	n_o	2
22	test right	1	n_o	2
23	Push right on S1	1	r_o	2
25	test left	1	n_o	2
26	Push left on S1	1	ℓ_o	2
	Total		**6n+3E+4**	**52**

$\ell_e + r_e + r_o + \ell_o = n - 1$. Check $42 + 6 + 4 = 52$. ✓

(d) Final complexity. $E \le n$, so $T(n)$ runs from about $6n + 1.5 \log_2 n$ (complete) to $7.5n$ (chain, worst count: every level adds outer-loop overhead). **Best = worst** $= \Theta(n)$. **Extra space** $\Theta(w)$: the stacks hold at most two levels.

Algorithm 8b Zigzag with a queue and reversal

Input: pointer root to the root
Output: same output as Algorithm 8a
Initial call: ZigzagQueue(root)

```
 1: procedure ZigzagQueue(root)
 2:     if root = NULL then
 3:         return
 4:     end if
 5:     Q ← empty queue
 6:     Enqueue(Q, root)
 7:     leftToRight ← true
 8:     while Q is not empty do
 9:         size ← Size(Q)
10:         level ← empty list
11:         for i ← 1 to size do
12:             node ← Dequeue(Q)
13:             Append(level, node.data)
14:             if node.left ≠ NULL then
15:                 Enqueue(Q, node.left)
16:             end if
17:             if node.right ≠ NULL then
18:                 Enqueue(Q, node.right)
19:             end if
20:         end for
21:         if leftToRight = false then
22:             lo ← 0; hi ← size −1
23:             while lo < hi do
24:                 Swap(level[lo], level[hi])
25:                 lo ← lo+1; hi ← hi−1
26:             end while
27:         end if
28:         for j ← 0 to size −1 do
29:             print level[j]
30:         end for
31:         leftToRight ← not leftToRight
32:     end while
33: end procedure
```

Twist: queue plus reversal. One row per level:

Lvl	Queue before	Dir.	level	Swaps	Printed	Queue after
1	5	L→R	5	0	5	12, 8
2	12, 8	R→L	12 8	1	8 12	21, 3, 17, 9
3	21, 3, 17, 9	L→R	21 3 17 9	0	21 3 17 9	empty

With O odd levels and P swaps ($\lfloor \text{size}/2 \rfloor$ per odd level): lines 2, 5, 6, 7 run once; line 8 $L+1$ times; lines 9, 10, 21, 31 and both for-inits L times; each for-test $n+L$ times; lines 12, 13, 14, 17, 29 and both increments n times; enqueues $n-1$; line 22 O; line 23 $P+O$; lines 24, 25 P each. Total

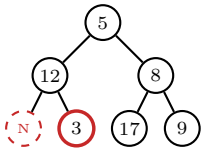
$$T_{8b}(n) = 10n + 9L + 2O + 3P + 4,$$

$$T_{8b}(7) = 70 + 27 + 2 + 3 + 4 = 106.$$

The two stacks do less work (52 vs. 106). They handle each node once and get the zigzag order for free; the queue version handles each node twice (into the list, then printed), spends 9 lines of bookkeeping per level, and 3 lines per swap (up to $n/2$ swaps). Both are $\Theta(n)$ time, $\Theta(w)$ space.

Question 9 (7 points) Is the tree complete?

Idea. Level order that enqueues *both* children, NULL or not. In a complete tree all real nodes precede all NULLs in level order (C6), so after the first NULL is dequeued, any real node is a gap: return false.



tree Q

(a) Pseudocode

Algorithm 9 Complete tree check

Input: pointer root to the root
Output: true iff the tree is complete
Initial call: COMPLETE(root)

```
 1: procedure COMPLETE(root)
 2:     if root = NULL then
 3:         return true
 4:     end if
 5:     Q ← empty queue
 6:     ENQUEUE(Q, root)
 7:     seenNull ← false
 8:     while Q is not empty do
 9:         node ← DEQUEUE(Q)
10:         if node = NULL then
11:             seenNull ← true
12:         else
13:             if seenNull = true then
14:                 return false                    ▷ gap
15:             end if
16:             ENQUEUE(Q, node.left)
17:             ENQUEUE(Q, node.right)
18:         end if
19:     end while
20:     return true
21: end procedure
```

(b) Worked example

One row per pass; N×k = k NULLs. Tree P:

Pass	Queue before	Deq.	Action	seen	Queue after
1	5	5	enq 12, 8	false	12, 8
2	12, 8	12	enq 21, 3	false	8, 21, 3
3	8, 21, 3	8	enq 17, 9	false	21, 3, 17, 9
4	21, 3, 17, 9	21	enq N, N	false	3, 17, 9, N, N
5	3, 17, 9, N, N	3	enq N, N	false	17, 9, N×4
6	17, 9, N×4	17	enq N, N	false	9, N×6
7	9, N×6	9	enq N, N	false	N×8
8	N×8	N	seenNull ← true	true	N×7
9	N×7	N	seenNull ← true	true	N×6
10	N×6	N	seenNull ← true	true	N×5
11	N×5	N	seenNull ← true	true	N×4
12	N×4	N	seenNull ← true	true	N×3
13	N×3	N	seenNull ← true	true	N, N
14	N, N	N	seenNull ← true	true	N
15	N	N	seenNull ← true	true	empty

16th test of line 8 finds the queue empty: COMPLETE(P) = **true**. Tree Q = $\{5, 12, 8, \text{NULL}, 3, 17, 9\}$:

Pass	Queue before	Deq.	Action	seen	Queue after
1	5	5	enq 12, 8	false	12, 8
2	12, 8	12	enq N, 3	false	8, N, 3
3	8, N, 3	8	enq 17, 9	false	N, 3, 17, 9
4	N, 3, 17, 9	N	seenNull ← true	true	3, 17, 9
5	3, 17, 9	3	**return false** (line 14)	true	—

COMPLETE(Q) = **false**. Both match the expected output.

(c) Frequency count

complete tree (worst case): $2n+1$ entries pass the queue, n real and $n+1$ NULL. The **else** has no cost.

Line	Operation	Cost	Freq.	P	Q
2	test root	1	1	1	1
5	Q ← empty	1	1	1	1
6	Enqueue(root)	1	1	1	1
7	seenNull ← false	1	1	1	1
8	test Q not empty	1	$2n$+2	16	5
9	Dequeue	1	$2n$+1	15	5
10	test NULL	1	$2n$+1	15	5
11	seenNull ← true	1	n+1	8	1
13	test seenNull	1	n	7	4
14	return false	1	0	0	1
16	Enqueue left	1	n	7	3
17	Enqueue right	1	n	7	3
20	return true	1	1	1	0
	Total		**10n+10**	**80**	**31**

(d) Final complexity.

Worst $\Theta(n)$: a complete tree needs every entry checked ($10n+10$). **Best** $\Theta(1)$: a gap under the root (root.left = NULL, root.right ≠ NULL) is caught on pass 3 after 19 lines. **Extra space** $\Theta(n)$: the queue ends with all $n+1$ NULLs (8 for P).

Twist. In level-order indices a tree is complete exactly when its nodes fill indices $0..m-1$ with no gap (C6). INSERT (I6) always uses index n, the first free slot, so $0..n$ stay filled. DELETE copies the deepest node (index $n-1$) into the target (D5) and removes index $n-1$ (D6), so $0..n-2$ stay filled. Both keep the indices contiguous, so the tree stays complete.

Question 10 (7 points) Lowest common ancestor

Idea (single pass). LCA(node) returns node if it holds a or b; otherwise it asks both subtrees. Both answers non-NULL means a and b lie on different sides, so node is the LCA; one non-NULL answer is passed up. If a is an ancestor of b the search stops at a, which is correct (a node is in its own subtree). Both values are assumed present.

(a) Pseudocode

Algorithm 10a LCA in one pass

Input: root; values a, b in the tree
Output: deepest node with a and b below it
Initial call: LCA(root, a, b)

```
 1: procedure LCA(node, a, b)
 2:     if node = NULL then
 3:         return NULL
 4:     end if
 5:     if node.data = a or node.data = b then
 6:         return node
 7:     end if
 8:     leftLCA ← LCA(node.left, a, b)
 9:     rightLCA ← LCA(node.right, a, b)
10:     if leftLCA ≠ NULL and rightLCA ≠ NULL then
11:         return node
12:     end if
13:     if leftLCA ≠ NULL then
14:         return leftLCA
15:     end if
16:     return rightLCA
17: end procedure
```

(b) Worked example

(L)/(R): passed up from the left/right subtree.

LCA(21, 9) = **5**:

#	Call	Lines	Returns
1	5	2,5,8–11	5 (split)
2	12	2,5,8–10,13,14	21 (L)
3	21	2,5,6	21 (match)
4	3	2,5,8–10,13,16	NULL
5	NULL (3.L)	2,3	NULL
6	NULL (3.R)	2,3	NULL
7	8	2,5,8–10,13,16	9 (R)
8	17	2,5,8–10,13,16	NULL
9	NULL (17.L)	2,3	NULL
10	NULL (17.R)	2,3	NULL
11	9	2,5,6	9 (match)

LCA(21, 3) = **12**:

#	Call	Lines	Returns
1	5	2,5,8–10,13,14	12 (L)
2	12	2,5,8–11	12 (split)
3	21	2,5,6	21 (match)
4	3	2,5,6	3 (match)
5	8	2,5,8–10,13,16	NULL
6	17	2,5,8–10,13,16	NULL
7	NULL (17.L)	2,3	NULL
8	NULL (17.R)	2,3	NULL
9	9	2,5,8–10,13,16	NULL
10	NULL (9.L)	2,3	NULL
11	NULL (9.R)	2,3	NULL

LCA(17, 8) = **8**; node 8 matches on line 5, so its subtree (with 17) is never searched:

#	Call	Lines	Returns
1	5	2,5,8–10,13,16	8 (R)
2	12	2,5,8–10,13,16	NULL
3	21	2,5,8–10,13,16	NULL
4	NULL (21.L)	2,3	NULL
5	NULL (21.R)	2,3	NULL
6	3	2,5,8–10,13,16	NULL
7	NULL (3.L)	2,3	NULL
8	NULL (3.R)	2,3	NULL
9	8	2,5,6	8 (match)

All three results match the expected output.

(c) Frequency count

m real nodes visited, r of them match (line 6), so $1+2(m-r)$ calls; $s = 1$ if line 11 runs; q nodes return via line 14.

Line	Operation	Cost	Frequency	(21,3)	(21,9)	(17,8)
2	test NULL	1	$1+2(m-r)$	11	11	9
3	return NULL	1	$m+1-2r$	4	4	4
5	test match	1	m	7	7	5
6	return node	1	r	2	2	1
8	left call	1	$m-r$	5	5	4
9	right call	1	$m-r$	5	5	4
10	test both	1	$m-r$	5	5	4
11	return node	1	s	1	1	0
13	test left	1	$m-r-s$	4	4	4
14	return leftLCA	1	q	1	1	0
16	return rightLCA	1	$m-r-s-q$	3	3	4
	Total		**$2+9m-8r-s$**	**48**	**48**	**39**

Worst case in n. If neither value is an ancestor of the other, both are visited ($r = 2$, $s = 1$), and $m = n$ when both are leaves: $T = 9n - 15$. If one is an ancestor of the other ($r = 1$, $s = 0$), at least the other is skipped, $m \le n-1$, $T = 9m-6 \le 9n-15$. So $T_{\text{worst}} = 9n - 15 = 48$ for $n = 7$, reached by LCA(21, 3) and LCA(21, 9). ✓

(d) Final complexity. **Worst** $\Theta(n)$ (a, b leaves: whole tree searched). **Best** $\Theta(1)$: a or b is the root (lines 2, 5, 6). **Extra space** $\Theta(h)$.

Algorithm 10b LCA with root-to-node paths

Input: root; values a, b in the tree
Output: the LCA of a and b
Initial call: LCAPaths(root, a, b)

```
 1: procedure FindPath(node, key, path)                ▷ path by ref.
 2:     if node = NULL then
 3:         return false
 4:     end if
 5:     Append(path, node)
 6:     if node.data = key then
 7:         return true
 8:     end if
 9:     if FindPath(node.left, key, path) = true then
10:         return true
11:     end if
12:     if FindPath(node.right, key, path) = true then
13:         return true
14:     end if
15:     RemoveLast(path)
16:     return false
17: end procedure
18: procedure LCAPaths(root, a, b)
19:     pA ← empty list; pB ← empty list
20:     FindPath(root, a, pA)
21:     FindPath(root, b, pB)
22:     i ← 0
23:     while i < min(|pA|, |pB|) and pA[i] = pB[i] do
24:         i ← i + 1
25:     end while
26:     return pA[i − 1]
27: end procedure
```

Twist: FINDPATH and compare. Each search visits a node at most once. With the key absent, FINDPATH runs line 2 $2n{+}1$ times, line 3 $n{+}1$ times and lines 5, 6, 9, 12, 15, 16 n times each: $9n + 2$; a successful search returns early and runs at most that. LCAPATHS adds lines 19–22 (4), at most $h{+}2$ tests on line 23, $h{+}1$ increments and line 26. Method 1 therefore costs up to $2(9n{+}2) + 2h + 8 = 18n + 2h + 12$, about twice method 2's $9n - 15$.

Query	pA / pB	FP(a)	FP(b)	own	M1	M2
(21, 3)	5,12,21 / 5,12,3	14	26	10	**50**	**48**
(21, 9)	5,12,21 / 5,8,9	14	56	8	**78**	**48**
(17, 8)	5,8,17 / 5,8	44	39	10	**93**	**39**

The last common path entry is the answer (12, 5, 8). Method 1 walks the shared upper part twice and stores two paths ($\Theta(h)$); method 2 visits each node at most once. Both $\Theta(n)$ worst case, $\Theta(h)$ space; the single pass does about half the work.

Question 11 (8 points) Build a tree from its preorder and inorder traversals

Idea. The next unused preorder value is the root of the range being built; its inorder position mid splits the range into in[lo..mid−1] (left subtree) and in[mid+1..hi] (right). Build left first, because preorder lists the whole left subtree before the right. A table pos[value] = inorder index gives mid in one step.

(a) Pseudocode

Algorithm 11 Build tree (lookup table)

Input: pre[0..n−1], in[0..n−1], distinct values
Output: root of the rebuilt tree
Initial call: root ← BuildTree(pre, in, n)

```
 1: procedure BuildTree(pre, in, n)
 2:     for k ← 0 to n − 1 do
 3:         pos[in[k]] ← k
 4:     end for
 5:     preIndex ← 0                                   ▷ shared
 6:     return Build(0, n − 1)
 7: end procedure
 8: procedure Build(lo, hi)
 9:     if lo > hi then
10:         return NULL
11:     end if
12:     node ← NewNode(pre[preIndex])
13:     preIndex ← preIndex +1
14:     mid ← pos[node.data]
15:     node.left ← Build(lo, mid −1)
16:     node.right ← Build(mid +1, hi)
17:     return node
18: end procedure
```

(b) Worked example

pre = 5 12 21 3 8 17 9, in = 21 12 3 5 17 8 9. Lines 2–4 build pos = {21:0, 12:1, 3:2, 5:3, 17:4, 8:5, 9:6} in 7 passes; then:

#	Build(lo, hi)	Lines	node; preIndex; mid	Ret.
1	(0, 6)	9,12–17	5; pI→1; mid 3	5
2	(0, 2)	9,12–17	12; pI→2; mid 1	12
3	(0, 0)	9,12–17	21; pI→3; mid 0	21
4	(0, −1)	9,10	–	NULL
5	(1, 0)	9,10	–	NULL
6	(2, 2)	9,12–17	3; pI→4; mid 2	3
7	(2, 1)	9,10	–	NULL
8	(3, 2)	9,10	–	NULL
9	(4, 6)	9,12–17	8; pI→5; mid 5	8
10	(4, 4)	9,12–17	17; pI→6; mid 4	17
11	(4, 3)	9,10	–	NULL
12	(5, 4)	9,10	–	NULL
13	(6, 6)	9,12–17	9; pI→7; mid 6	9
14	(6, 5)	9,10	–	NULL
15	(7, 6)	9,10	–	NULL

Result = the tree from arr: level order **5, 12, 8, 21, 3, 17, 9**, postorder **21, 3, 12, 17, 9, 8, 5** = expected.

(c) Frequency count Build runs $2n+1$ times.

Line	Operation	Cost	Freq.	Count
2	for: init / test / incr.	1	1 / n+1 / n	1/8/7
3	pos[in[k]] ← k	1	n	7
5	preIndex ← 0	1	1	1
6	return Build(0, n−1)	1	1	1
9	test lo > hi	1	$2n+1$	15
10	return NULL	1	$n+1$	8
12	NewNode	1	n	7
13	advance preIndex	1	n	7
14	mid ← pos[·]	1	n	7
15	left call	1	n	7
16	right call	1	n	7
17	return node	1	n	7
	Total		**12n + 6**	**90**

(d) Final complexity. $12n+6 = \Theta(n)$, **best** = **worst**: no row depends on the shape. **Extra space** $\Theta(n)$ (table) + $\Theta(h)$ (stack).

Algorithm 11b Build tree (linear search)

Input: same arrays as Algorithm 11
Output: root of the rebuilt tree
Initial call: root ← BuildTreeSearch(pre, in, n)

```
 1: procedure BuildTreeSearch(pre, in, n)
 2:     preIndex ← 0
 3:     return BuildS(0, n − 1)
 4: end procedure
 5: procedure BuildS(lo, hi)
 6:     if lo > hi then
 7:         return NULL
 8:     end if
 9:     node ← NewNode(pre[preIndex])
10:     preIndex ← preIndex +1
11:     mid ← lo
12:     while in[mid] ≠ node.data do
13:         mid ← mid +1
14:     end while
15:     node.left ← BuildS(lo, mid −1)
16:     node.right ← BuildS(mid +1, hi)
17:     return node
18: end procedure
```

Twist: searching the inorder list. Let $S = \sum(\text{mid} - \text{lo})$, the positions skipped by all searches. Line 12 runs $n+S$ times and line 13 S times; the rest keep their counts (lines 2, 3: 1; 6: $2n$+1; 7: n+1; 9, 10, 11, 15, 16, 17: n each):

$$T_{\text{search}}(n) = 10n + 4 + 2S.$$

Here $S = 3+1+1 = 5$ (only roots 5, 12, 8 are not first in their range), $T = 84$. **Worst case**: a left chain, where each root is the *last* value of its range, so $S = 0+1+\cdots+(n-1) = n(n-1)/2$ and

$$T_{\text{worst}}(n) = n^2 + 9n + 4 = \Theta(n^2) \quad (116 \text{ for } n = 7).$$

Best: a right chain ($S = 0$, $10n+4$). The lookup table replaces the search loop (lines 11–14) by one line at a one-time cost of $3n+2$ lines (lines 2–3), so every row is $O(n)$ and $T = 12n+6 = \Theta(n)$ for every shape. On our small balanced tree the search is slightly cheaper (84 vs. 90); on the 7-node left chain the table wins (90 vs. 116), and the gap grows quadratically.

Question 12 (8 points) Diameter of a binary tree

Idea. The longest path bending at a node has $(lh+1)+(rh+1) = lh+rh+2$ edges (lh, rh subtree heights, height(NULL) = −1). The diameter is the largest such value. One postorder pass returns each height (as in Q1) and updates best on the way up.

(a) Pseudocode

Algorithm 12 Diameter in one pass

Input: pointer root to the root
Output: edges on the longest path between two nodes
Initial call: Diameter(root)

```
 1: procedure Diameter(root)
 2:     best ← 0
 3:     HeightDia(root, best)
 4:     return best
 5: end procedure
 6: procedure HeightDia(node, best)                ▷ best by ref.
 7:     if node = NULL then
 8:         return −1
 9:     end if
10:     lh ← HeightDia(node.left, best)
11:     rh ← HeightDia(node.right, best)
12:     if lh + rh +2 > best then
13:         best ← lh + rh +2
14:     end if
15:     return 1 + max(lh, rh)
16: end procedure
```

(b) Worked example best shown when the node finishes.

#	Call	Lines	lh+rh+2	best	Ret.
1	5	7,10–13,15	$1+1+2=4$	2 → **4**	2
2	12	7,10–13,15	$0+0+2=2$	0 → **2**	1
3	21	7,10–12,15	$-1-1+2=0$	0	0
4	NULL (21.L)	7,8	–	–	−1
5	NULL (21.R)	7,8	–	–	−1
6	3	7,10–12,15	$-1-1+2=0$	0	0
7	NULL (3.L)	7,8	–	–	−1
8	NULL (3.R)	7,8	–	–	−1
9	8	7,10–12,15	$0+0+2=2$	2	1
10	17	7,10–12,15	$-1-1+2=0$	2	0
11	NULL (17.L)	7,8	–	–	−1
12	NULL (17.R)	7,8	–	–	−1
13	9	7,10–12,15	$-1-1+2=0$	2	0
14	NULL (9.L)	7,8	–	–	−1
15	NULL (9.R)	7,8	–	–	−1

best: 0 → 2 at 12, 2 → 4 at 5. Output **4** (path 21, 12, 5, 8, 17) = expected.

(c) Frequency count u = updates of best ($u = 2$; $0 \le u \le n$).

Line	Operation	Cost	Freq.	Count
2	best ← 0	1	1	1
3	HeightDia(root)	1	1	1
4	return best	1	1	1
7	test node = NULL	1	$2n+1$	15
8	return -1	1	$n+1$	8
10	left call	1	n	7
11	right call	1	n	7
12	test $lh{+}rh{+}2 >$ best	1	n	7
13	best ← $lh{+}rh{+}2$	1	u	2
15	return height	1	n	7
	Total		$\mathbf{7n+5+u}$	**56**

(d) Final complexity. $7n+5 \le T(n) \le 8n+5$, so **best** = **worst** = $\Theta(n)$. Largest count: a chain (best grows at every node above the leaf, $u = n-1$, $T = 8n+4$). **Extra space** $\Theta(h)$.

Algorithm 12b Diameter calling MaxDepth at every node

Input: pointer node to the root
Output: the diameter
Initial call: DiaNaive(root)

```
1: procedure DiaNaive(node)
2:     if node = NULL then
3:         return 0
4:     end if
5:     thr ← MaxDepth(node.left) + MaxDepth(node.right) +2
6:     dl ← DiaNaive(node.left)
7:     dr ← DiaNaive(node.right)
8:     return max(thr, dl, dr)
9: end procedure
```

Twist: the $\Theta(n^2)$ solution vs. one pass. DiaNaive's own lines run $(2n{+}1){+}(n{+}1){+}4n = 7n{+}2$ times. Line 5 also runs MaxDepth (Algorithm 1, $6m+2$ lines on m nodes) on both children: $6(m_v-1)+4$ at a node with subtree size m_v. Since $\sum_v (m_v-1) = D$, the total depth of all nodes (each node is counted once per ancestor),

$$T_{\text{naive}}(n) = 11n + 2 + 6D.$$

Our tree: $D = 0+2+8 = 10$, $T = 139$ (vs. 56). A chain: $D = n(n-1)/2$, $T = 3n^2+8n+2 = \Theta(n^2)$ (205 for $n = 7$). Algorithm 12 never recomputes a height: each call returns it to its parent, every line has frequency at most $2n+1$, so $7n+5+u \le 8n+5 = \Theta(n)$.

Question 13 (8 points) Serialize and deserialize a binary tree

Idea. Write the tree in preorder with N for every NULL child. The N markers show where each subtree ends, so reading the list in the same preorder rebuilds the same shape (no inorder list needed). A shared index idx tells DESERIALIZE which item to read next.

(a) Pseudocode

Algorithm 13a Serialize

Input: root node; list out (by reference, empty)
Output: out = preorder values with N for NULL
Initial call: Serialize(root, out)

```
1: procedure Serialize(node, out)
2:     if node = NULL then
3:         Append(out, N)
4:         return
5:     end if
6:     Append(out, node.data)
7:     Serialize(node.left, out)
8:     Serialize(node.right, out)
9: end procedure
```

Algorithm 13b Deserialize

Input: a Serialize list; idx (by reference)
Output: root of the rebuilt tree
Initial call: idx ← 0; root ← Deserialize(list, idx)

```
1: procedure Deserialize(list, idx)
2:     if list[idx] = N then
3:         idx ← idx +1
4:         return NULL
5:     end if
6:     node ← NewNode(list[idx])
7:     idx ← idx +1
8:     node.left ← Deserialize(list, idx)
9:     node.right ← Deserialize(list, idx)
10:    return node
11: end procedure
```

(b) Worked example Both visit positions in the same preorder: call k of Serialize writes item k, call k of Deserialize reads it.

#	Position	Serialize lines	Appends	Deserialize lines	idx	Deserialize returns
1	5	2,6–8	5	2,6–10	$0 \to 1$	node 5
2	12	2,6–8	12	2,6–10	$1 \to 2$	node 12
3	21	2,6–8	21	2,6–10	$2 \to 3$	node 21
4	NULL (21.L)	2–4	N	2–4	$3 \to 4$	NULL
5	NULL (21.R)	2–4	N	2–4	$4 \to 5$	NULL
6	3	2,6–8	3	2,6–10	$5 \to 6$	node 3
7	NULL (3.L)	2–4	N	2–4	$6 \to 7$	NULL
8	NULL (3.R)	2–4	N	2–4	$7 \to 8$	NULL
9	8	2,6–8	8	2,6–10	$8 \to 9$	node 8
10	17	2,6–8	17	2,6–10	$9 \to 10$	node 17
11	NULL (17.L)	2–4	N	2–4	$10 \to 11$	NULL
12	NULL (17.R)	2–4	N	2–4	$11 \to 12$	NULL
13	9	2,6–8	9	2,6–10	$12 \to 13$	node 9
14	NULL (9.L)	2–4	N	2–4	$13 \to 14$	NULL
15	NULL (9.R)	2–4	N	2–4	$14 \to 15$	NULL

Serialize output **5 12 21 N N 3 N N 8 17 N N 9 N N** (15 items) = expected; Deserialize returns the tree with level order 5, 12, 8, 21, 3, 17, 9 (the tree from arr).

(c) Frequency count Serialize:

Line	Operation	Cost	Freq.	Count
2	test node = NULL	1	$2n+1$	15
3	Append N	1	$n+1$	8
4	return	1	$n+1$	8
6	Append data	1	n	7
7	left call	1	n	7
8	right call	1	n	7
	Total		$\mathbf{7n+3}$	**52**

Deserialize:

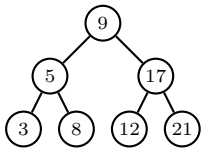
Line	Operation	Cost	Freq.	Count
2	test list[idx] = N	1	$2n+1$	15
3	idx ← idx +1	1	$n+1$	8
4	return NULL	1	$n+1$	8
6	NewNode	1	n	7
7	idx ← idx +1	1	n	7
8	left call	1	n	7
9	right call	1	n	7
10	return node	1	n	7
	Total		$\mathbf{9n+3}$	**66**

Twist ($2n+1$ items). One item per value (n) plus one N per NULL pointer ($2n$ pointers $-$ $(n-1)$ edges $= n+1$): $2n+1$ items (15). Each call of either procedure handles exactly one item, so both run $2n+1$ times (line 2 in both tables); the N branch runs $n+1$ times, the value branch n times.

(d) Final complexity. $7n+3$ and $9n+3$: both $\Theta(n)$, **best = worst**, since the list always has $2n+1$ items. **Extra space** $\Theta(h)$; the list is the output.

Question 14 (8 points) Is the tree a valid binary search tree?

Idea. Each node must lie strictly inside the range (low, high) set by its ancestors: going left lowers high to the node's value, going right raises low. Checking only parent–child pairs is not enough: in 10 (5 (2, 12), 15) every pair is in order, but 12 is in the left subtree of 10.



tree S

(a) Pseudocode

Algorithm 14a BST check with ranges

Input: node; open range (low, high)
Output: true iff the subtree is a valid BST in range
Initial call: IsBST(root, $-\infty$, $+\infty$)

```
 1: procedure IsBST(node, low, high)
 2:     if node = NULL then
 3:         return true
 4:     end if
 5:     if node.data ≤ low or node.data ≥ high then
 6:         return false
 7:     end if
 8:     if IsBST(node.left, low, node.data) = false then
 9:         return false
10:     end if
11:     return IsBST(node.right, node.data, high)
12: end procedure
```

(b) Worked example Tree P: 12 is the left child of 5 but not below 5.

#	Call	Range	Lines	Returns
1	5	$(-\infty, +\infty)$	2,5,8,9	false
2	12	$(-\infty, 5)$	2,5,6	**false** (12 out of range)

Tree S: every node is inside its range.

#	Call	Range	Lines	Returns
1	9	$(-\infty, +\infty)$	2,5,8,11	true
2	5	$(-\infty, 9)$	2,5,8,11	true
3	3	$(-\infty, 5)$	2,5,8,11	true
4	NULL (3.L)	$(-\infty, 3)$	2,3	true
5	NULL (3.R)	$(3, 5)$	2,3	true
6	8	$(5, 9)$	2,5,8,11	true
7	NULL (8.L)	$(5, 8)$	2,3	true
8	NULL (8.R)	$(8, 9)$	2,3	true
9	17	$(9, +\infty)$	2,5,8,11	true
10	12	$(9, 17)$	2,5,8,11	true
11	NULL (12.L)	$(9, 12)$	2,3	true
12	NULL (12.R)	$(12, 17)$	2,3	true
13	21	$(17, +\infty)$	2,5,8,11	true
14	NULL (21.L)	$(17, 21)$	2,3	true
15	NULL (21.R)	$(21, +\infty)$	2,3	true

ISBST(P) = **false**, ISBST(S) = **true** = expected.

(c) Frequency count worst case = valid BST.

Line	Operation	Cost	Freq.	S	P
2	test node = NULL	1	$2n+1$	15	2
3	return true	1	$n+1$	8	0
5	test out of range	1	n	7	2
6	return false	1	0	0	1
8	test left subtree	1	n	7	1
9	return false	1	0	0	1
11	return right subtree	1	n	7	0
	Total		$\mathbf{6n+2}$	**44**	**7**

Algorithm 14b BST check with inorder

Input: node; prev (by reference)
Output: true iff inorder is strictly increasing
Initial call: prev $\leftarrow -\infty$; IsBSTIn(root, prev)

```
 1: procedure IsBSTIn(node, prev)
 2:     if node = NULL then
 3:         return true
 4:     end if
 5:     if IsBSTIn(node.left, prev) = false then
 6:         return false
 7:     end if
 8:     if node.data ≤ prev then
 9:         return false
10:     end if
11:     prev ← node.data
12:     return IsBSTIn(node.right, prev)
13: end procedure
```

(d) Final complexity. **Worst** $\Theta(n)$ ($6n+2$, any valid BST). **Best** $\Theta(1)$: a violation at the root's left child is caught on call 2 (P: 7 lines). **Extra space** $\Theta(h)$.

Twist: inorder solution. A BST's inorder is strictly increasing, so compare each node with the previous value. On P the first comparison is only at the leftmost node 21; the next, $12 \le 21$, fails:

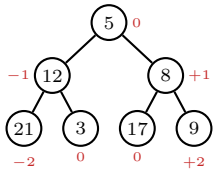
#	Call	Lines	prev	Returns
1	5	2,5,6	—	false
2	12	2,5,8,9	$12 \le 21$	**false**
3	21	2,5,8,11,12	$-\infty \to 21$	true
4	NULL (21.L)	2,3	—	true
5	NULL (21.R)	2,3	—	true

On S, prev runs $-\infty \to 3 \to 5 \to 8 \to 9 \to 12 \to 17 \to 21$: true. Valid BST: line 2 $2n+1$, line 3 $n+1$, lines 5, 8, 11, 12 n each, lines 6, 9 zero, so $T = 7n+2$ (51 for S); 16 on P.

Comparison. Worst case both $\Theta(n)$: $6n+2$ vs. $7n+2$ (inorder spends one more line per node on prev). Best cases differ: ranges can reject at the root's left child, $\Theta(1)$ (7 on P); inorder makes its first comparison only after reaching the leftmost node, so its earliest rejection costs $\Theta(d)$, d = depth of the leftmost node (16 on P).

Question 15 (8 points) Vertical order traversal

Idea. Level order on (node, distance) pairs; append each value to its column's list. Level order is top to bottom, left to right, so each column is already in the required order. Track minD and maxD; every column between them is non-empty (distance changes by 1 per edge).



(a) Pseudocode

Algorithm 15 Vertical order traversal

Input: pointer root to the root
Output: columns from leftmost to rightmost distance
Initial call: VERTICAL(root)

```
 1: procedure VERTICAL(root)
 2:     if root = NULL then
 3:         return
 4:     end if
 5:     cols ← empty map distance → list
 6:     minD ← 0; maxD ← 0
 7:     Q ← empty queue                          ▷ (node, d) pairs
 8:     ENQUEUE(Q, (root, 0))
 9:     while Q is not empty do
10:         (node, d) ← DEQUEUE(Q)
11:         APPEND(cols[d], node.data)
12:         if d < minD then
13:             minD ← d
14:         end if
15:         if d > maxD then
16:             maxD ← d
17:         end if
18:         if node.left ≠ NULL then
19:             ENQUEUE(Q, (node.left, d−1))
20:         end if
21:         if node.right ≠ NULL then
22:             ENQUEUE(Q, (node.right, d+1))
23:         end if
24:     end while
25:     for d ← minD to maxD do
26:         for j ← 0 to SIZE(cols[d]) −1 do
27:             print cols[d][j]
28:         end for
29:     end for
30: end procedure
```

(b) Worked example

One row per pass; v_d = value v at distance d.

Pass	Queue before	Deq.	cols[d] (line 11)	13/16	Queue after
1	5_0	5_0	[0]=[5]	—	12_{-1} 8_{+1}
2	12_{-1} 8_{+1}	12_{-1}	[−1]=[12]	min←−1	8_{+1} 21_{-2} 3_0
3	8_{+1} 21_{-2} 3_0	8_{+1}	[+1]=[8]	max←+1	21_{-2} 3_0 17_0 9_{+2}
4	21_{-2} 3_0 17_0 9_{+2}	21_{-2}	[−2]=[21]	min←−2	3_0 17_0 9_{+2}
5	3_0 17_0 9_{+2}	3_0	[0]=[5, 3]	—	17_0 9_{+2}
6	17_0 9_{+2}	17_0	[0]=[5, 3, 17]	—	9_{+2}
7	9_{+2}	9_{+2}	[+2]=[9]	max←+2	empty

Lines 25–29 then print d = −2..+2: **−2: 21** | **−1: 12** | **0: 5, 3, 17** | **+1: 8** | **+2: 9** = expected (3 before 17 by level order).

(c) Frequency count

c = maxD − minD +1 columns ($c = 5$); a, b updates on lines 13, 16 (2, 2). A new minimum is always one below the old (its parent was already seen), likewise the maximum, so $a + b = c - 1$.

Line	Operation	Cost	Freq.	Count
2	test root	1	1	1
5	cols ← empty	1	1	1
6	minD, maxD ← 0	1	1	1
7	Q ← empty	1	1	1
8	Enqueue(root, 0)	1	1	1
9	test Q not empty	1	$n+1$	8
10	Dequeue	1	n	7
11	Append to cols[d]	1	n	7
12	test d < minD	1	n	7
13	minD ← d	1	a	2
15	test d > maxD	1	n	7
16	maxD ← d	1	b	2
18	test left	1	n	7
19	Enqueue left	1	n_L	3
21	test right	1	n	7
22	Enqueue right	1	n_R	3
25	for d: init / test / incr.	1	1 / c+1 / c	1/6/5
26	for j: init / test / incr.	1	c / n+c / n	5/12/7
27	print	1	n	7
	Total		**11n+4c+a+b+7**	**108**

With $n_L + n_R = n - 1$, $a + b = c - 1$: $T = 11n + 5c + 6$; check $77 + 25 + 6 = 108$. ✓

(d) Final complexity.

$2 \le c \le n$ (for $n \ge 2$), so $11n + 16 \le T(n) \le 16n + 6$: **best** = **worst** = $\Theta(n)$. Worst count: a one-sided chain ($c = n$); best: a zigzag chain ($c = 2$). Grouping costs one line per node (hash map, or an array of $2n - 1$ lists indexed by $d + n - 1$); printing costs $3n + 4c + 2$. **Extra space** $\Theta(n)$ (columns hold all values; queue up to w pairs).

Twist (number of columns). Each column holds at least one node, so $c \le n$, reached by a chain that always goes the same way (distances $0, -1, \ldots, -(n-1)$). Distances can range over $2n - 1$ values, $-(n-1)..n-1$, but at most n are occupied. A single node has 1 column, a zigzag chain only 2, and a complete tree of height h has $2h + 1$ (5 here).